Initial Parallelization and Scaling Baseline

OpenMP Partitioned and Parallel Matrix (PPM) decoding for SD erasure codes — gPPM project, Milestone 2

Executive Summary

Milestone 1 delivered a validated, scalar, single-threaded SD-code encoder/decoder (`ec_recover`). Milestone 2 parallelizes recovery with the PPM idea from the gPPM paper [1,3]: stripe rows whose failures are self-contained are decoded as **independent groups** by OpenMP threads, and the remaining **dependent** failures are decoded afterwards. The sequential baseline is left untouched and is used as the reference for both correctness and speed.

Correctness. PPM output is byte-identical to the sequential baseline in all **450/450** timed benchmark trials (9 configurations × baseline + T=1,2,4,8 × 10 trials; one checksum per configuration) and in **7,833/7,833** layouts of the full SD parameter grid. All unit tests pass.

Performance. At 8 threads the decoder is **1.18×–2.33×** faster than the sequential baseline (median 1.49×; best m=3, s=1, GF(2^8)). Of that, 1.02×–1.24× is already obtained at one thread from the algorithmic work reduction; pure thread scaling adds 1.13×–1.96×.

Plateau. Scaling flattens at 4–8 threads because only about 22–66% of the work is parallel (the dependent remainder is serial: Amdahl's law) and only 15 independent tasks exist per stripe, which divide unevenly over 8 threads (Section 6.4).

Milestone 2 requirement	Where addressed
Parallel implementation (OpenMP)	Section 1; `src/ppm.c`, `src/ppm.h`
Correctness comparison with sequential version	Section 3; `ppm_benchmark_results.csv`, `ppm_sweep_results.csv`
Initial performance and scaling results	Sections 4 and 5
Why OpenMP, not GPU; number of threads; advantages / disadvantages	Sections 6.1 – 6.3
Why performance plateaus as threads increase	Section 6.4
Memory requirement of the sequential baseline (Milestone 1)	Section 6.5

1. Parallel Implementation

1.1 Where the parallelism comes from

An SD code $SD^{m,s}_{n,r}$ has a parity-check matrix H with $m{\cdot}r$ sparse, row-local rows (m equations per stripe row) and s dense rows that span the whole stripe. If a stripe row contains exactly m failed sectors, its m local equations determine them completely, without touching any other row's data. Such a row is an **independent group**. Rows with fewer or more failures, plus the s dense equations, form the **dependent remainder** H_{rest}, which needs the independently recovered sectors as input and therefore must run afterwards [1,3].

For the benchmark layout (n=16, r=16, z=1: the s extra sector failures fall in one row) there are **p = r − z = 15 independent groups** and $m{\cdot}z + s$ dependent failures (2–6).

1.2 Algorithm and OpenMP structure

Step	What happens	Parallel?
1. Classify	Count failures per stripe row; rows with exactly m failures become independent groups	serial (tiny)
2. Prepare	For each group: extract its m rows of H, split into F and S, invert F (m×m). Same for H_{rest}	serial
3. Independent recovery	`#pragma omp parallel num_threads(min(T,p))` + `omp for schedule(static)` over the p groups; each thread runs `ec_decode_matrix_first` ($G = F^{-1}S$, then $B_F = G \cdot B_S$)	**parallel**
4. Barrier	Implicit barrier at the end of the parallel region; per-thread operation counters are summed	sync
5. Remainder	`ec_decode_normal` on H_{rest} using the sectors recovered in step 3	serial

- **Race-free by construction.** Row-local equations only involve columns of their own stripe row, so each group reads only its own surviving sectors and writes only its own faulty sectors. Groups share no data, so no locks or atomics are needed.
- **Lock-free instrumentation.** The operation counter `gf_mult_xors_count` is `_Thread_local`; each thread counts its own `mult_XORs` calls and the totals are added after the barrier, so the hot path has no synchronization.
- **Sequential path unchanged.** `ec_recover()` (single thread, normal sequence) is not modified and serves as the reference. `ppm_recover_sd()` is a separate entry point taking the requested thread count.
- **Exact arithmetic.** Galois-field multiply/XOR is integer arithmetic, so results do not depend on the order of operations: the parallel output must be bit-identical to the sequential output, which makes byte comparison a strict correctness test (unlike floating-point reductions).

1.3 Two sources of speedup

PPM is faster than the baseline for two separate reasons, and this report keeps them apart. (i) **Work reduction**: partitioning plus matrix-first order lowers the number of full-sector multiply-XOR operations C (e.g. 527 → 497 for m=1, s=1), visible already at T=1. (ii) **Thread parallelism**: the independent groups run concurrently, visible as improvement from T=1 to T=8. “Speedup” below means baseline time ÷ PPM time (both effects); “self-speedup” means PPM(T=1) time ÷ PPM(T) time (thread scaling only).

2. Experimental Method

Item	Setting
Platform (as recorded for Milestone 1)	Intel Core Ultra 9 185H, GCC 11.4.0, `-O2 -fopenmp`, scalar code (no SIMD)
Workload	SD(n=16, r=16), 32 MiB codeword, 128 KiB sectors, z = 1; (m,s) ∈ {1,2,3}², field $GF(2^8)/GF(2^{16})/GF(2^{32})$ as m,s grow (FAST coefficients)
Failure layout	Deterministic worst case: m whole disks + s extra sectors in one row (identical for baseline and PPM)
Variants timed	Sequential baseline; PPM with T = 1, 2, 4, 8 requested threads (`schedule(static)`)
Timing	`CLOCK_MONOTONIC` around the complete recovery call only (includes matrix split/inversion); 1 warm-up + 10 trials per point; **median** reported; validation is outside the timed region
Data	`results/ppm_benchmark_results.csv` (450 trials), `results/ppm_sweep_results.csv` (7,833 layouts)

Throughput is reported as useful (non-parity) data per second, the same convention as Milestone 1. Scaling numbers were measured on the recorded multi-core platform; the correctness checks were additionally re-executed independently for this report (Section 3.3).

3. Correctness Comparison with the Sequential Version

Every PPM result is compared against the sequential baseline using five independent checks. A trial passes only if all hold.

Check	Criterion	Result
Byte equality	Recovered 32 MiB codeword is `memcmp`-identical to the original encoded codeword	450 / 450
Checksum equality	64-bit FNV-1a checksum of PPM output equals baseline output checksum; exactly one distinct checksum per configuration over its 50 trials (baseline + 4 thread counts)	9 / 9 configs
Syndrome	$H \times B = 0$ over the whole recovered stripe	450 / 450
Operation count	Measured C = independent + remainder counts; PPM C ≤ baseline C; C identical for every thread count	450 / 450
Full parameter grid	`ppm_sweep`: 3,969 published SD configurations, 7,833 feasible (config, z) layouts at T=1 and T=4: bytes equal, syndrome zero, count equal across T	7,833 / 7,833

3.1 Per-configuration comparison

m	s	Field	Baseline C	PPM C	Work saved	Trials passed	Output checksum (baseline = PPM, all T)
1	1	GF(2^8)	527	497	5.7%	50 / 50	`32245b01c79a59cb`
1	2	GF(2^{16})	783	753	3.8%	50 / 50	`6100d6a8bb694ebb`
1	3	GF(2^{32})	1039	1009	2.9%	50 / 50	`253b0b42a3723b6b`
2	1	GF(2^8)	828	708	14.5%	50 / 50	`7280f9f79ad51ad7`
2	2	GF(2^{16})	1084	964	11.1%	50 / 50	`4faad3f7034343e7`
2	3	GF(2^{32})	1340	1220	9.0%	50 / 50	`00554160eb4964c3`
3	1	GF(2^8)	1159	889	23.3%	50 / 50	`0c9d9237eb6f1c47`
3	2	GF(2^{16})	1415	1145	19.1%	50 / 50	`54573baccfacb34b`
3	3	GF(2^{32})	1671	1401	16.2%	50 / 50	`3c9b5192b0f83a57`

Table 1. C = number of full-sector multiply-XOR operations. One checksum per row confirms the sequential and all parallel runs produced the same bytes.

3.2 Full-grid partition and work check

Across all 7,833 feasible layouts PPM finds between 1 and 23 independent groups (7,770 layouts use more than one thread at T=4) and needs fewer multiply-XORs than the baseline in every one: mean reduction **15.8%** (min 1.0%, max 70.1%). Operation counts at T=1 and T=4 are identical in all layouts, showing that thread count changes only *when* work runs, never *what* is computed.

3.3 Independent re-execution for this report

- `make test` and `make ppm-test` (GF arithmetic, SD construction and round trips, coefficient table, PPM partitioning and OpenMP recovery): **all PASS**; the paper's Figure 2 example still gives C = 35 for the baseline and the Figure 3 example gives C = 29 for PPM.
- `make ppm-sweep` re-run from scratch: 7,938 points, 105 infeasible, **7,833 passed, 0 failed**, and the output CSV is identical to the recorded one.
- The PPM benchmark was re-run on a **single-CPU Linux container** (GCC 13.3) with 1, 2, 4 and 8 threads deliberately **oversubscribed** on one core: all 45 validated trials PASS. Forcing threads to interleave arbitrarily is a strong test that correctness does not depend on timing or core count. (No speed results are taken from that run.)

4. Performance Results

Median decode time and speedup over the sequential baseline (32 MiB codeword, 10 trials per point):

m	s	Field	Base (ms)	PPM T=1 (ms)	PPM T=2 (ms)	PPM T=4 (ms)	PPM T=8 (ms)	Speedup T=1	Speedup T=2	Speedup T=4	Speedup T=8
1	1	GF(2^8)	40.1	35.5	30.6	29.9	28.5	1.13	1.31	1.34	1.41
1	2	GF(2^{16})	50.9	49.2	46.2	43.0	41.8	1.03	1.10	1.18	1.22
1	3	GF(2^{32})	47.4	45.6	41.6	39.6	40.3	1.04	1.14	1.20	1.18
2	1	GF(2^8)	68.7	67.2	54.9	41.7	38.8	1.02	1.25	1.65	1.77
2	2	GF(2^{16})	77.7	71.7	57.6	52.4	53.3	1.08	1.35	1.48	1.46
2	3	GF(2^{32})	62.2	53.2	45.6	44.9	41.7	1.17	1.36	1.39	1.49
3	1	GF(2^8)	101.4	85.2	61.1	48.2	43.5	1.19	1.66	2.10	2.33
3	2	GF(2^{16})	101.5	85.2	66.3	56.5	60.9	1.19	1.53	1.80	1.67
3	3	GF(2^{32})	78.0	62.7	50.9	47.5	45.1	1.24	1.53	1.64	1.73

Table 2. Median time and speedup (baseline time ÷ PPM time). T=1 already includes the algorithmic work reduction.

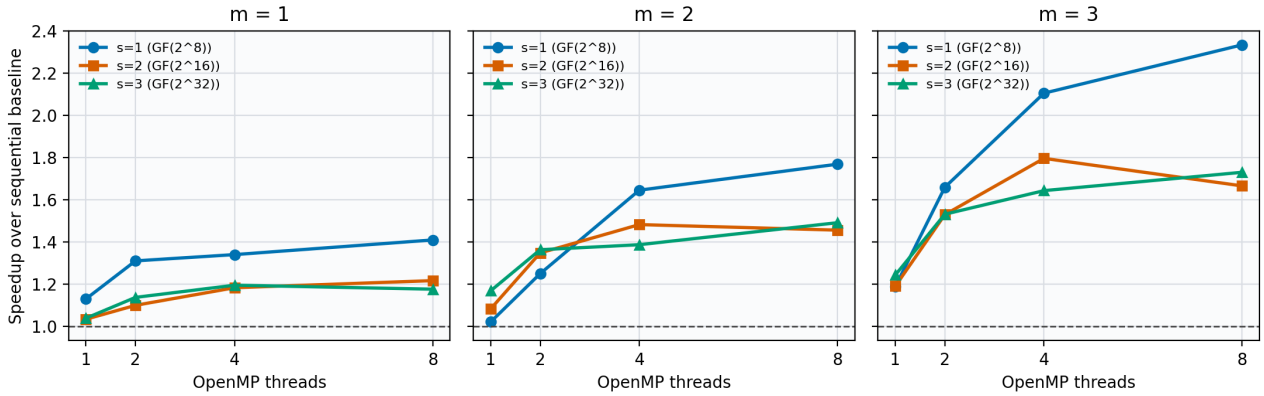


Figure 1. Speedup over the sequential baseline versus OpenMP threads. Dashed line: baseline (1×).

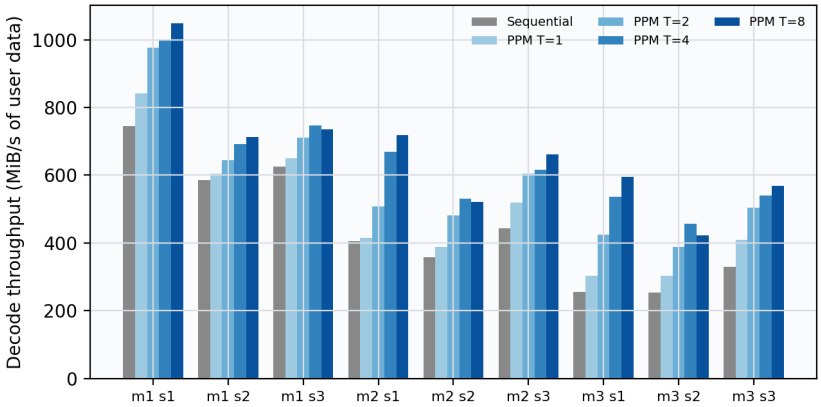


Figure 2. Decode throughput (useful data) for the sequential baseline and PPM at each thread count.

5. Scaling Results

Table 3 isolates thread scaling: self-speedup = PPM(T=1) time ÷ PPM(T) time, and parallel efficiency = self-speedup ÷ T.

m	s	Field	Indep. groups	Speedup T=2	Speedup T=4	Speedup T=8	Effic. T=2	Effic. T=4	Effic. T=8
1	1	GF(2^8)	15	1.16	1.19	1.25	58%	30%	16%
1	2	GF(2^{16})	15	1.06	1.14	1.18	53%	29%	15%
1	3	GF(2^{32})	15	1.09	1.15	1.13	55%	29%	14%
2	1	GF(2^8)	15	1.22	1.61	1.73	61%	40%	22%
2	2	GF(2^{16})	15	1.24	1.37	1.34	62%	34%	17%
2	3	GF(2^{32})	15	1.17	1.19	1.28	58%	30%	16%
3	1	GF(2^8)	15	1.39	1.77	1.96	70%	44%	25%
3	2	GF(2^{16})	15	1.28	1.51	1.40	64%	38%	17%
3	3	GF(2^{32})	15	1.23	1.32	1.39	62%	33%	17%

Table 3. Thread scaling only (excludes work reduction): speedup = self-speedup = PPM(T=1) time ÷ PPM(T) time; efficiency = speedup ÷ T.

Observations

- Scaling is **sub-linear and saturating**: at T=8 self-speedup is only 1.13×–1.96× (efficiency 14–25%).
- The gain from T=1 to T=2 is the largest. Going from T=4 to T=8 adds only 3–11% in six configurations, and in three ((m,s) = (1,3), (2,2), (3,2)) T=4 is actually faster than T=8.
- Configurations with larger *m* and smaller *s* scale best (m=3, s=1: 1.96× at T=8) because they have the largest independent share of the work (Figure 3).
- **Sweet spot:** T=4 already reaches at least 90% of the best speedup in every configuration; T=8 helps mainly m ≥ 2 with GF(2^8).

6. Discussion and Answers to the Milestone Questions

6.1 Why OpenMP, and why not a GPU?

Why OpenMP.

- **The parallelism is coarse task parallelism with a fork–join shape.** PPM exposes 15 independent groups, each performing hundreds of operations on 128 KiB regions, followed by a dependent remainder. That is precisely `parallel for` + implicit barrier + serial tail.
- **Minimal, low-risk change to validated code.** The Milestone 1 C code is reused as-is; the parallel layer is one directive-based region and the program still compiles and runs sequentially without `-fopenmp`. The existing correctness harness applies unchanged.
- **Shared memory, no data movement.** The codeword already sits in host RAM (in a storage node it arrives from NIC/NVMe into host memory), so threads operate on it in place. A GPU would need the 32 MiB stripe copied to the device and back over PCIe (roughly 64 MiB per decode; a few ms at PCIe 4.0 ×16 speeds — not prohibitive compared with the 28–100 ms decode times, but extra latency and complexity).
- **The current field arithmetic is table-driven.** GF(2^8/2^{16}) use log/antilog lookups and GF(2^{32}) uses a 4 MiB split table. Such irregular gather accesses are cheap on a CPU cache but do not fit in GPU shared memory (tens of KiB per SM), so a GPU port would first require re-formulating the multiply (e.g. bit-matrix/XOR or shared-memory tables), which is a separate research task.
- **Portability and scope.** OpenMP needs only a C compiler. CUDA requires an NVIDIA GPU, while the recorded platform is an Intel CPU. The reference gPPM/PPM work parallelizes on multicore CPUs [1,3], so OpenMP gives the like-for-like scaling baseline this milestone asks for.

Why not GPU (now). Not because a GPU could never help: region multiply-XOR over MiB-sized buffers is data-parallel, and a GPU would also attack the serial remainder that limits this design. It is deferred because it needs a different arithmetic kernel, adds transfer cost, and cannot be validated against the

CPU code as directly. It is listed as future work.

6.2 Number of parallel threads

Quantity	Value
Thread counts evaluated	T = 1, 2, 4, 8 (requested via `num_threads(T)`)
Maximum useful parallelism	p = r − z = **15** independent groups per stripe (n=16, r=16, z=1). Effective threads = min(T, p); asking for more than 15 adds no speedup
Work distribution	`schedule(static)`: T=2 → 8+7 groups, T=4 → 4+4+4+3, T=8 → seven threads with 2 groups and one with 1
Hardware available	Reference CPU Intel Core Ultra 9 185H: 16 cores / 22 hardware threads (hybrid P- and E-cores), so T ≤ 8 does not oversubscribe the machine
Recommended setting	**T = 4** as default (≥ 90% of best speedup everywhere); T = 8 for m ≥ 2 with GF(2^8)

6.3 Advantages and disadvantages of OpenMP for this project

Advantages	Disadvantages
Directive-based; incremental change to working C code; sequential build still possible	Limited to the cores of one machine; distributed storage nodes would need MPI or a hybrid design
Shared address space: no copies or messages; groups read and write the stripe in place	Speedup is bounded by the number of independent tasks (15) and by the serial remainder
Very low fork/join overhead relative to 28–100 ms decode times	`schedule(static)` with few tasks gives load imbalance when T does not divide 15
Race-free design here (disjoint writes) and lock-free per-thread counters	Threads share caches and memory bandwidth; no control over hybrid P/E-core placement unless pinned
Portable across GCC/Clang/MSVC and operating systems; no special hardware	Lower peak throughput than a GPU for massively data-parallel work
Easy to vary T for scaling studies (Section 5)	Timing varies with OS scheduling, turbo and thermal behaviour; data races are easy to introduce if the design is extended

6.4 Why performance plateaus as threads increase

Three effects combine; the first dominates. The measurements are consistent with Amdahl's law [6]: speedup(T) = 1 / ((1 − f) + f/T), where f is the parallelizable fraction.

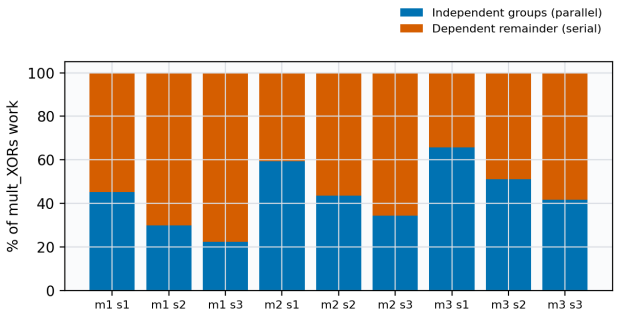


Figure 3. Share of multiply-XOR work in the parallel independent groups versus the serial dependent remainder (T=1 operation counts).

- **1. A serial section that threads cannot shorten (Amdahl).** The dependent remainder H_{rest} must wait for the independent groups and then runs on one thread. It is 34–78% of the operations (Figure 3), and it grows with s and shrinks (relatively) with m. Even with infinitely many threads speedup is capped at 1/(1 − f): about 1.3× for m=1, s=1 and 2.3× for m=3, s=1 (fitted from timings, Table 4). The serial preparation (splitting and inverting matrices) adds a little more.
- **2. Only 15 tasks, distributed unevenly.** With a static schedule the parallel section can speed up by at most 15/⌈15/T⌉: 1.88× at T=2, 3.75× at T=4 and 7.5× at T=8. At T=8 one thread has only one group and idles while seven do two. More threads than 15 cannot help at all.

- **3. Shared resources and hardware effects (not separately measured).** Threads share L2/L3 cache and memory bandwidth, and on a hybrid CPU some threads land on slower E-cores. These plausibly explain why the measured points sit below the Amdahl bound and why a few configurations get marginally slower from T=4 to T=8 (e.g. m=3, s=2: 1.80× → 1.67×). Verifying them would need hardware counters and thread pinning, which we have not done.

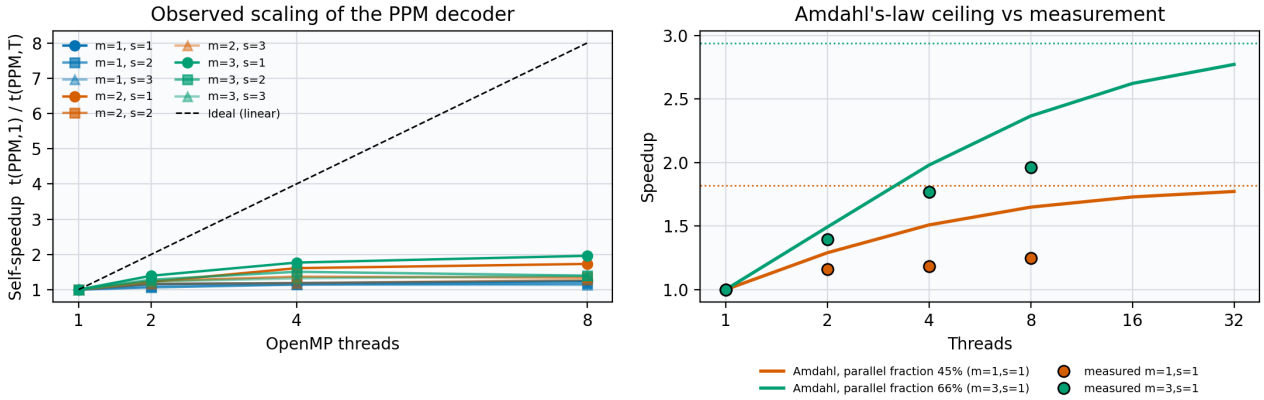


Figure 4. Left: observed self-speedup of all nine configurations against the ideal line. Right: Amdahl curves (from operation-count fractions 45% and 66%) against measured points; dotted lines are the asymptotic ceilings 1/(1 − f).

m	s	Field	f from operation counts	Amdahl ceiling 1/(1−f)	f fitted from timings	Ceiling (fitted)	Measured self-speedup T=8
1	1	GF(2^8)	45%	1.83×	23%	1.29×	1.25×
1	2	GF(2^{16})	30%	1.43×	16%	1.19×	1.18×
1	3	GF(2^{32})	22%	1.29×	15%	1.18×	1.13×
2	1	GF(2^8)	59%	2.46×	47%	1.89×	1.73×
2	2	GF(2^{16})	44%	1.77×	33%	1.50×	1.34×
2	3	GF(2^{32})	34%	1.52×	24%	1.32×	1.28×
3	1	GF(2^8)	66%	2.92×	57%	2.31×	1.96×
3	2	GF(2^{16})	51%	2.04×	39%	1.63×	1.40×
3	3	GF(2^{32})	42%	1.72×	33%	1.49×	1.39×

Table 4. Parallel fraction f = independent multiply-XORs ÷ total multiply-XORs (operation-count estimate) and f fitted by least squares to the measured self-speedups at T = 1, 2, 4, 8. The fitted f is lower because serial preparation, per-operation cost differences and hardware effects are not visible in operation counts. Measured speedup never exceeds the operation-count Amdahl bound.

Consequence. Adding threads to the current task structure has diminishing returns. The next gains must come from shrinking the serial part: split each region operation of the remainder across threads by byte range, use SIMD in `mult_XORs`, and choose the cheaper calculation sequence per matrix (gPPM, Milestone 3).

6.5 Memory requirement of the sequential baseline (Milestone 1)

The sequential decoder works in place on the codeword and allocates only a scratch area and small matrices. Sizes below are for the benchmark geometry (n=16, r=16 → 256 sectors of 128 KiB); $R_H = m \cdot r + s$ is the number of parity equations.

Component	Formula	m=1, s=1	m=3, s=3 (worst case)
Codeword stripe (in place; recovered sectors overwrite failed ones)	n·r·sector	32.00 MiB	32.00 MiB
Scratch $T = S \cdot B_S$ (normal sequence)	R_H·sector	2.13 MiB (17 sectors)	6.38 MiB (51 sectors)
Matrices H, F, S, F^{-1}, Gauss-Jordan [F|I] (32-bit entries)	$4 \cdot (2 \cdot R_H \cdot n \cdot r + 3 \cdot R_H^2)$ bytes	≈ 0.04 MiB	≈ 0.13 MiB
GF tables	field-dependent	$GF(2^8)$: 3 KiB	$GF(2^{32})$: 4 MiB (16 planes × 64 Ki × 4 B); $GF(2^{16})$: 768 KiB
Syndrome accumulator (validation)	1 sector	0.13 MiB	0.13 MiB
Minimum for a production decoder		**≈ 34.3 MiB**	**≈ 42.6 MiB**
Benchmark harness: golden reference copy for validation	stripe size	+ 32 MiB	+ 32 MiB
Measured peak resident set (whole benchmark process)			**77.2 MiB**

Table 5. Peak resident memory was measured with `getrusage(RUSAGE_CHILDREN)` *on* `sd_benchmark` *(all nine configurations, 1 trial each, all three field tables loaded, includes the golden copy and C runtime): 77.2 MiB. The PPM benchmark peaks at 77.9 MiB, so parallelization does not materially change memory use.*

- Memory is dominated by the **codeword itself** (n·r·sector = 32 MiB); the scratch buffer and tables add only 7% (m=1, s=1) to 33% (m=3, s=3, $GF(2^{32})$) on top of it. The requirement therefore scales linearly with stripe size and is independent of thread count.
- The scratch buffer grows with the number of parity equations (m·r + s), so larger m or s costs more memory; the matrices are negligible.
- PPM needs *no extra sector buffers*: independent groups write directly into the stripe (matrix-first order needs no T), and the remainder's scratch is only (m·z + s) sectors — 0.25 MiB instead of 2.13 MiB for m=1, s=1. Extra memory is limited to small per-group matrices.
- The 32 MiB codeword exceeds a typical shared L3 cache, but a 128 KiB sector fits in L2. Region operations are therefore streaming accesses over cache-resident sectors, which is why the memory footprint is not the bottleneck of the parallel scaling.

7. Limitations and Next Steps

- Scaling data comes from one machine, one codeword size (32 MiB), one deterministic failure layout (z = 1) and T ≤ 8; the reported speedups are therefore an **initial** scaling baseline.
- Speedups are modest (best 2.33× at T=8) because the serial remainder is large; this is a structural limit of the current PPM decomposition, not an implementation defect.
- **Next:** (1) parallelize the remainder by byte-range decomposition of each region operation; (2) SIMD in `mult_XORs` (Milestone 3); (3) gPPM dynamic selection between normal and matrix-first sequences using C = u + c·v; (4) try `schedule(dynamic)` and thread pinning; (5) evaluate other z values, sector sizes and T up to 15; (6) explore a GPU region kernel.

Reproducing the results

`make ppm-test` · `make ppm-sweep` · `make ppm-benchmark` · `make plots` (README.md documents each target).

References

[1] S. Li, Q. Cao, S. Wan, W. Xia, and C. Xie, "gPPM: A Generalized Matrix Operation and Parallel Algorithm to Accelerate the Encoding/Decoding Process of Erasure Codes," ACM Transactions on Architecture and Code Optimization, vol. 20, no. 4, art. 51, 2023.

[2] J. S. Plank, M. Blaum, and J. L. Hafner, “SD Codes: Erasure Codes Designed for How Storage Systems Really Fail,” USENIX FAST, 2013.

[3] S. Li et al., “PPM: A Partitioned and Parallel Matrix Algorithm to Accelerate Encoding/Decoding Process of Asymmetric Parity Erasure Codes,” ICPP, 2015.

[4] J. S. Plank, K. M. Greenan, and E. L. Miller, “Screaming Fast Galois Field Arithmetic Using Intel SIMD Instructions,” USENIX FAST, 2013.

[5] OpenMP Architecture Review Board, “OpenMP Application Programming Interface, Version 5.2,” November 2021.

[6] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” AFIPS Spring Joint Computer Conference, 1967.

[7] P. Sobe, “Parallel Reed/Solomon Coding on Multicore Processors,” Int. Workshop on Storage Network Architecture and Parallel I/Os, 2010.